Kern

Migration and Code Alignment Guide

v0.1 -> v0.2

Java 25+ | RocksDB | Spring Boot | Armeria | gRPC

Group ID: it.riccisi.kern

Document purpose

This guide is an executable refactoring plan for code already started from the previous specification. It is intentionally explicit enough to be supplied to Codex or another coding agent together with the repository.

> **Migration rule**
> Do not patch the existing key-revision DCB implementation incrementally until both models coexist. Introduce the v0.2 query model behind new types, migrate callers and tests, then delete the obsolete path.

1. Executive summary of the breaking change

v0.1	v0.2
Event primarily belongs to one stream/subject	Event belongs to global log and may carry many tags
DCB uses Map<ConsistencyKey, revision>	DCB uses EventQuery + global observedAt
Conflict when any consistency key revision changed	Conflict only when a later event matches types AND tags in the query
Per-key revision/watermark is authoritative	Global position and event indices are authoritative
Classic and DCB append are separate concepts	Classic Event Sourcing is an adapter over single-tag query semantics
Tag watermark can decide conflicts	Watermarks may only accelerate negative checks, never decide full queries

2. What can remain unchanged

- Maven multi-module layout and groupId it.riccisi.kern.
- Spring Boot bootstrap and Armeria transport choice.
- Single logical AppendCoordinator and bounded queue.
- RocksDB lifecycle, WAL durability and WriteBatch strategy.
- Global event positions and the authoritative events column family.
- Idempotency, event-ID uniqueness, backup, recovery and control plane.
- Subscription transport based on global positions.
- Observability infrastructure, although DCB-specific attributes must change.

3. What must be removed or deprecated

Element	Action	Reason
ConsistencyKey	Remove from public API; optionally retain internal migration utility	Not the DCB consistency primitive
ConsistencyExpectation(key, revision)	Delete	Replaced by query and observed position
DcbCondition(Map<key, revision>)	Delete	Produces false conflicts and cannot express type filtering
consistency column family	Drop after migration	No authoritative per-key revisions
tag-watermark conflict detector	Delete as decision logic	Cannot represent type + multi-tag predicates
Event.stream as mandatory ownership	Make optional adapter input or remove	An event may participate in many histories
supportsKeyBasedDcb capability	Replace with	Terminology and semantics changed

Element	Action	Reason
	supportsQueryConditionalAppend	

4. New domain/API types to introduce first

```
public record EventTag(String name, String value) {}
public record EventType(String value) {}
public record SequencePosition(long value) {
    public SequencePosition next() { return new SequencePosition(value + 1); }
}

public record QueryItem(
        Set<EventType> types,
        Set<EventTag> tags
) {}

public record EventQuery(List<QueryItem> items) {}

public record AppendCondition(
        EventQuery failIfEventsMatch,
        SequencePosition after
) {}

public record QueryResult(
        List<SequencedEvent> events,
        SequencePosition observedAt,
        ContinuationToken continuation
) {}
```

4.1 EventData change

```
// Before
record EventData(UUID id, String type, String stream,
                 byte[] payload, byte[] metadata) {}

// After
record EventData(EventId id, EventType type,
                 Set<EventTag> tags, ContentType contentType,
                 byte[] payload, byte[] metadata) {}
```

If the existing client API exposes stream, keep it temporarily as a facade that injects a reserved or conventional tag. Do not persist a separate authoritative stream ownership field unless required for compatibility metadata.

5. Storage migration

5.1 Column-family changes

Existing	Target	Migration
events	events	Keep; update event codec to include tags and format version
streams	tag_index or classic adapter index	Convert stream ID to one event tag
subjects/tags	tag_index	Normalize into tag + position posting list
types	type_index	Keep or re-encode with namespace/type/position
consistency	Removed	Do not copy
tag_watermarks	Optional cache only	Rebuild or discard
idempotency	idempotency	Keep; version fingerprint algorithm

Existing	Target	Migration
metadata	metadata	Add schema and key-codec version

5.2 Development database policy

Because the existing database is not production data, the preferred migration is destructive: bump the storage format version, delete the local database directory and recreate it. Avoid writing a complex online migration for disposable development data.

```
if (storedFormatVersion < FORMAT_V2) {
    throw new IncompatibleDatabaseFormatException(
        "Delete the development database or run kern migrate --from-v1");
}
```

5.3 Optional fixture converter

If existing test fixtures are valuable, implement an offline converter that maps stream to a tag and rewrites event records. It must not infer DCB consistency from old key revisions.

6. Query engine implementation

1. Implement QueryItem matching in memory and property-test it.
2. Implement scans for type-only, tag-only and tag+type items.
3. Implement union of QueryItems ordered by global position with de-duplication.
4. Implement multi-tag verification against candidate events.
5. Return observedAt from metadata using the same RocksDB snapshot.
6. Add findFirst(query, after, head) for append conflict detection.
7. Only after correctness, add query planner statistics and selectivity choices.

```
interface QueryEngine {
    List<SequencedEvent> execute(EventQuery query,
                                 SequencePosition fromExclusive,
                                 SequencePosition toInclusive,
                                 int limit,
                                 ReadOptions options);

    Optional<SequencedEvent> findFirst(EventQuery query,
                                       SequencePosition fromInclusive,
                                       SequencePosition toInclusive,
                                       ReadOptions options);
}
```

7. AppendCoordinator refactoring

7.1 Before

```
verifyExpectedStreamRevision(request.stream(), request.expectedRevision());
verifyConsistencyRevisions(request.consistency());
writeEventsAndIncrementRevisions();
```

7.2 After

```
verifyIdempotency(request);
Optional<SequencedEvent> conflict = queryEngine.findFirst(
    request.condition().failIfEventsMatch(),
    request.condition().after().next(),
    currentHead,
    overlayReadOptions
);
if (conflict.isPresent()) rejectWithExplanation(conflict.get());
else appendEventsAndIndices(request.events());
```

7.3 Group-commit overlay

This is a high-risk area. If multiple requests share a physical batch, request B must see events accepted for request A in the same uncommitted group. Otherwise two conflicting commands could both pass.

- Simplest safe first implementation: commit one logical request per WriteBatch.
- Second step: add an in-memory overlay of newly assigned events and evaluate later queries against RocksDB plus overlay.
- Never enable group commit across logical requests until a race test proves this property.

8. Classic stream adapter

Preserve a familiar API without maintaining a second consistency engine.

```
public CompletionStage<AppendResult> appendToStream(
        StreamId stream,
        SequencePosition observedAt,
        List<EventData> events,
        IdempotencyKey key) {

    EventTag streamTag = new EventTag("$stream", stream.value());
    EventQuery query = new EventQuery(List.of(
        new QueryItem(Set.of(), Set.of(streamTag))
    ));

    List<EventData> tagged = events.stream()
        .map(event -> event.withAdditionalTag(streamTag))
        .toList();

    return append(new AppendRequest(
        tagged, new AppendCondition(query, observedAt), key));
}
```

If callers currently know only expected stream revision, the read API can return both event count and observedAt. Revision remains presentation metadata; observedAt is used for the actual append condition.

9. Protocol changes

Message	Required change
Event	Replace stream/subject ownership with repeated tags
ReadRequest	Add repeated QueryItem and position bounds
ReadResponse	Add observed_at global position
AppendRequest	Replace consistency revisions with AppendCondition query + after
Conflict response	Return invalidating event position/type/tags and matched item
Capabilities	Advertise QUERY_CONDITIONAL_APPEND and

Message	Required change
	CLASSIC_STREAM_ADAPTER

```
message QueryItem {
  repeated string event_types = 1;
  repeated EventTag tags = 2;
}
message EventQuery { repeated QueryItem items = 1; }
message AppendCondition {
  EventQuery fail_if_events_match = 1;
  uint64 after = 2;
}
```

10. Observability changes

Old attribute/metric	Replacement
consistency.keys	query.items, query.tags, query.types
expected_revision / actual_revision	observed_at / conflicting_position
consistency_conflicts_total	query_conflicts_total by matched type/tag
tag revision lookup time	query planning and candidate scan time
conflicting key	conflicting event + matched query item

- Never log payload by default.
- Persist or expose a normalized representation of the append condition.
- Trace candidate count, selected index and scan range.
- Add conflict age = conflicting event time - observation time where metadata permits.

11. Test migration matrix

Old test	Action
increments consistency key revisions	Delete
rejects when any tag changed	Replace with type-aware cases
event belongs to one stream	Replace with one stored event visible through multiple tag queries
expected stream revision conflict	Keep through classic adapter
atomic event + indexes	Keep and expand for all tags/types
idempotent retry	Keep; fingerprint includes query condition

11.1 New must-have tests

- StudentAddressChanged does not invalidate enrolment query.
- CourseDescriptionChanged does not invalidate enrolment query.
- StudentEnrolled for same course invalidates query.
- StudentEnrolled for same student invalidates query.
- One StudentEnrolled event is returned by both student and course queries.
- A QueryItem with two tags matches only events containing both.
- observedAt may exceed last matching position without causing false conflicts.
- Snapshot prevents observedAt from including an event omitted from results.
- Group batch later request sees earlier uncommitted matching event.

12. Recommended implementation sequence for Codex

Step	Scope	Exit condition
1	Introduce v0.2 API types without removing old code	New types compile and have unit tests
2	Change event codec and development format version	Fresh DB writes/reads multi-tag events
3	Build tag/type indices and QueryEngine	Read conformance tests pass
4	Return snapshot-consistent observedAt	Race test passes
5	Add query conditional append, initially one request per batch	DCB race tests pass
6	Implement classic stream adapter	Old stream use cases pass through new engine
7	Change gRPC/protobuf and client	End-to-end tests pass
8	Change diagnostics and metrics	Conflict explanation test passes
9	Delete key-revision APIs and consistency CF	No references remain
10	Reintroduce safe group commit with overlay	Batch race and benchmark gates pass

13. Codex execution prompt

The following prompt can be supplied with this document and the repository:

```
Refactor Kern from the v0.1 key-revision DCB model to the v0.2
query-based model described in the attached migration guide.

Work incrementally. For each step:
1. inspect the current repository and list affected files;
2. implement only the current migration step;
3. add or update tests before deleting old behavior;
4. run the full Maven test suite;
5. report API breaks, storage-format breaks and remaining obsolete code.

Do not preserve ConsistencyKey, ConsistencyExpectation or per-key
revision checks in the public model. DCB consistency is exclusively:
EventQuery + observed global SequencePosition. QueryItem types and tags
must both participate in matching. Traditional stream behavior must be
implemented as an adapter using one stream tag and no type filter.

Do not enable multi-request group commit until an overlay test proves
that later requests observe earlier accepted events in the same batch.
Do not add Spring or transport dependencies to kern-api or kern-core.
```

14. Completion checklist

- No public ConsistencyKey or revision-map classes remain.
- No authoritative consistency/tag-watermark column family remains.
- Every event codec supports multiple tags.
- Every read returns snapshot-consistent observedAt.
- Append conditions contain the exact read query.
- Conflict detection evaluates type and all required tags.
- Classic stream operations use the same conditional append engine.
- Diagnostics expose the invalidating event and matched item.

- All new race tests pass repeatedly.
- README and specification references point to Kern v0.2.

Appendix A - File search terms for the existing repository

```
ConsistencyKey
ConsistencyExpectation
DcbCondition
expectedRevision
streamRevision
consistency_revision
consistency column family
tagWatermark
supportsKeyBasedDcb
RevisionObservation
Map<ConsistencyKey
```

Search these terms before starting. Classify each result as remove, adapt, compatibility facade or unrelated.